



Wildfire Drone Deployment Simulation – Game Design & Technical Plan

Introduction

Project Overview: *Wildfire Drone Deployment* is a 3D simulation game (built in Unity 6.2 URP) that lets players combat real-world wildfires using autonomous firefighting drones. It leverages live NASA Earth data on active fires and a global 3D map to create a realistic, strategy-driven experience. Players assume the role of an international commander, monitoring a **spinning 3D globe** with active wildfire hotspots, then deploying drone fleets from various countries' defense forces to extinguish fires and save lives. The game blends a **real-time strategy (RTS)** overview (global map with drone fleet management) with an **action mode** (third-person drone control for water/retardant drops). The goal is to put out fires efficiently under time pressure, maximizing lives and property saved (measured in points and monetary value).

Hackathon Context: This prototype is built for the NDIA 2025 Hackathon, aligning with multiple focus areas: *Geospatial Analytics* (global fire mapping), *Autonomous Unmanned Systems* (drone fleets), *Modeling & Simulation* (wildfire spread and suppression), and *Contested Logistics* (managing limited resources under pressure). It also showcases how defense technology can support disaster response, addressing **Mission Impact** by tackling a critical real-world problem. The design balances **realism** (using NASA's near real-time wildfire data) with **practicality** (fast data streaming and simple controls), given the 1–3 day prototype timeframe. The target platform is PC (with a possible WebGL build for browser play via iframe), tested on Windows 11.

Gameplay Design

1. Gameplay Overview

In **Global Command View (RTS mode)**, the player sees a 3D Earth with active fires marked. Fires are sourced from NASA's Fire Information for Resource Management System (FIRMS) and updated in near real-time (within ~3 hours of satellite observation ¹). The player can rotate/zoom the globe, click on fire icons to view fire details (location, intensity, etc.), and select a nation's firefighting drone unit to deploy. Choosing a country (e.g. USA, Brazil, Australia) will send that nation's drones – each country's drones are **visually distinct** (unique color scheme and label) for authenticity. For now, differences are cosmetic (e.g. U.S. drones might appear in gray with USAF markings, Brazil's in green with a flag decal, etc.), since implementing varied models per country is beyond the rapid prototype scope. The globe visualization must be *realistic but lightweight*: we will likely use a **streamed 3D earth** (via an API like Cesium for Unity) or a static textured Earth model for performance. Cesium's Unity plugin provides a full-scale high-accuracy WGS84 globe and can stream global terrain and imagery ², which aligns with our need for an easy, realistic globe solution. (Unity 6.2 URP is compatible with Cesium for Unity, which supports URP ³.) Alternatively, a static NASA Blue Marble texture on a sphere could be used for simplicity.

In **Tactical Drone View (3rd Person action mode)**, the player can zoom in and take direct control of one drone in a fleet. This mode is entered by selecting an active drone team and switching camera to follow a specific drone. The player can then steer the drone in real-time and trigger water or fire retardant drops on the blaze. The *action* element gives a close-up perspective – e.g. flying through smoke to make a precision drop on a fire hotspot. We'll use Unity's physics to simulate water/retardant drops (particles or projectiles) and fire suppression. When in drone view, the rest of the world/time still runs (fires may grow), so the player must balance high-level strategy with hands-on firefighting.

2. Core Mechanics

- **Active Fire Data Integration:** The simulation ingests live or recent wildfire data from NASA FIRMS. FIRMS provides near real-time active fire “hotspot” locations detected by satellites (MODIS and VIIRS) ¹. For prototyping, we can pull the *last 24 hours of global fires* via FIRMS web services (available as CSV, JSON, KML, or shapefiles ⁴). Each fire data point includes latitude/longitude, brightness, and other attributes. These will spawn as fire markers on the globe. If real-time streaming is too slow or large, we will use a **historical snapshot** (e.g. fires from a notable week in 2025) to ensure quick loading. The design goal is to have *at least hundreds of fire points* visible globally, demonstrating scale, but we may filter or cluster them to avoid overwhelming the player. For example, minor fires could be grouped or omitted, focusing on major incidents.
- **Deploying Drone Fleets:** The player can deploy a limited number of **drone fleets** at any time. A fleet represents a group of firefighting drones launched together to tackle a fire or fire cluster. For game balance, suppose the player has e.g. **3 fleets available** (this can be tuned for difficulty). Each fleet contains multiple drones (e.g. 5 drones) that work in unison on one fire incident. Deploying a fleet is done by selecting a country and then clicking a target fire on the map. This triggers an animation of drones “launching” – for instance, icons might fly from the country's location or an aircraft carrier icon toward the fire location. (In a polished version, we'd show an actual carrier aircraft or base; for the prototype, a simple line or drone icons moving on the globe suffices.)

Deployment realism: In reality, militaries could deploy drones via cargo planes or mobile command centers. In-game, each country is assumed to have at least one **deployment base** or **carrier**. For authenticity we note that large transport aircraft or naval carriers could carry multiple drones – e.g. the U.S. Air Force could deploy several UAVs from a C-130 or a drone “mothership” concept (a notion even DARPA has explored for military swarms ⁵). We'll simplify this by giving each country a notional “carrier” capacity: e.g. each deployment can launch up to 5 drones simultaneously. The player cannot exceed the total drone count or fleets in the air at once, simulating limited resources. If a fleet is busy or a drone is refueling (e.g. after a water drop run), the player must wait or recall that fleet before redeploying. This introduces **contested logistics** gameplay – choosing where to send your finite drones first is critical.

- **Fire Suppression:** Each fire on the map has a variable intensity/size. When a drone fleet arrives at a fire, an **extinguishing timer** begins for that fire. One drone fleet might take, say, 30 in-game seconds to neutralize a small fire, but several minutes for a large one. Players can choose to **stack multiple fleets** on a major fire to put it out faster – but then fewer resources are available elsewhere. Fires may also *spread or grow* over time if left unattended (for simplicity, we might not simulate complex spread, but we can simulate increasing intensity or affected area over the timed session). In action mode, the player can help extinguish faster by manually directing drops on the most intense fire spots – a direct hit could dramatically reduce the remaining fire strength bar. If the

player ignores a fire for too long, it might “flare up” (e.g. requiring more drones or time to extinguish, imitating growth).

- **Switching Views and Multi-Tasking:** The player can always toggle between the globe view and any active drone’s POV. While micromanaging one drone in third-person, the other drones in that fleet will continue to operate under AI (circling and dousing the fire). The player must keep an eye on the big picture – e.g., an alert system can warn if a new large fire appears or if another fire is nearing a populated area – prompting the player to exit drone view and deploy/reassign fleets as needed. This tests the player’s **situational awareness** (a realistic challenge for incident commanders).
- **Time Limit and Win Condition:** The game session is time-bound (e.g. a **10-minute round** to simulate an intense initial attack period). The objective is to extinguish as many fires as possible (or all fires) before time runs out, minimizing damage. A *score* is calculated based on performance (details in Scoring section below). The game ends when the timer elapses, and the player is presented with a **mission summary**: fires put out vs. total, lives saved, property saved in \$\$\$, and an overall score (with a rank or medal, e.g. “Gold – Exceptional Response” if they score above a threshold). There’s no “failure” state except scoring low – even if fires remain, the round ends and the impact of the remaining fires is considered “damage”.

3. Fire Data and Map Implementation

Global Map: To render the Earth, we have two main options: (a) **Cesium for Unity** – a powerful plugin that streams real-world 3D geospatial content (terrain, satellite imagery, buildings) into Unity ²; or (b) a simpler static globe. **Cesium for Unity** is attractive for realism: it provides a high-accuracy globe and can pull base imagery (like Bing Maps or Cesium’s own satellite tiles) and elevation data via the Cesium ion service ². This would allow our prototype to have a zoomable Earth with actual terrain relief and potentially even 3D city models. Given our tight time, we might primarily use Cesium for the base terrain/imagery. (We note that Cesium’s plugin is open-source and integrates with Unity’s GameObjects and coordinates ⁶, making it feasible to place our fire markers by latitude/longitude directly.) The **Universal Render Pipeline (URP)** is supported by Cesium for Unity ³, so we can maintain URP for WebGL build compatibility. One concern is build size/performance for WebGL – streaming high-res 3D tiles might be heavy. If WebGL performance is an issue, plan B is using a **textured sphere**. We can obtain NASA’s Blue Marble 2km global texture (a few MBs) or use a static tile layer from NASA’s GIBS (Global Imagery Browse Services) as the globe texture ⁷ ⁸. This would be less dynamic but very lightweight – ideal for an embedded web demo. The globe will also have an overlay for night-time lights or simply an unlit side to simulate day/night if we include a time-of-day cycle.

Fire Markers: Each active fire from FIRMS data is represented as a marker (e.g. a small flame icon or glowing dot) at the correct geolocation on the globe. If using Cesium/real coordinates, conversion from lat/long to Unity world position is handled by the geospatial plugin. If we use a custom sphere, we’ll calculate positions via spherical coordinates: e.g. $x = R \cos(\text{lat}) \cos(\text{lon})$, $y = R \sin(\text{lat})$, $z = R \cos(\text{lat}) \sin(\text{lon})$ for a sphere radius R . We might scale the altitude of markers slightly above the surface to avoid z-fighting with the terrain. The marker color or size could indicate intensity (e.g. a big red flame icon for high-intensity fire, smaller orange for lower intensity). We may allow a *filter** toggle (e.g. show only major fires or only certain regions) to manage clutter. Additionally, integrating NASA’s near-real-time fire imagery is possible: NASA FIRMS provides a Web Map Service (WMS) with active fires which could be displayed as an overlay layer (with an API key) ⁹ ¹⁰. However, for simplicity, individual markers updated from the CSV feed is enough.

NASA FIRMS Data Access: We will utilize FIRMS' API endpoints or data downloads for active fires. NASA FIRMS offers **open web services** for active fire data – one can request the last 24 hours of fire detections globally, after obtaining a free API “MAP_KEY” via NASA's Earthdata login ⁹. The API provides data in multiple formats; for Unity, CSV or JSON is easiest. For example, a CSV might list each fire with fields: latitude, longitude, brightness, time, confidence, etc. ⁴. We will retrieve this at game start (using Unity's `UnityWebRequest` or an HTTP GET) and parse the results to generate the fire GameObjects. Since Earthdata API has rate limits (5000 requests per 10 minutes ¹¹), we will cache data as needed and not spam requests. If real-time updating is needed (e.g. new fires during gameplay), we might periodically refresh, but in a short round it may not be necessary. **Earthdata Account:** The developer will register for an Earthdata account (if not already) to get the API key ¹⁰. This ensures we abide by NASA's usage policies while accessing the data in near real-time.

Google Maps for Unity: Another option considered is the Google Maps Platform. Google had a Unity SDK for real-world maps (used in games like Pokémon GO) that provides realistic 3D city models and geospatial data. However, Google's Unity mapping SDK is *not openly available/free* to all developers ¹² – it requires access to the Google Maps Platform for gaming and involves usage costs and quotas. Given our focus on wildfires in often remote areas (forests, etc.), the benefit of Google's detailed urban data is limited. Instead, using open data (NASA/Cesium) is more practical and cost-effective. If needed, we could use Google's **static map APIs** (e.g. to fetch an image of an area) or leverage Google Earth KML integration (since FIRMS KML can be viewed in Google Earth). But for our game's 3D interactive globe, Google's solutions are either unavailable or redundant. For completeness, we note the existence of **ArcGIS Maps SDK for Unity** as well – a plugin by Esri that streams real-world map layers and has global data ¹³. It's an alternative to Cesium, offering high-quality basemaps and even analytic services (e.g. routing, if we needed). We may not need ArcGIS for the prototype, but it's good to know multiple geospatial Unity SDKs exist.

4. Drone Units and Equipment

Drone Types: All firefighting drones in the game are assumed to be **unmanned aerial vehicles (UAVs)** capable of carrying firefighting payloads. In reality, drones used in wildfire management range from small quadcopters for surveillance to large military-grade drones (like the MQ-9 Reaper) adapted to drop retardant or monitor fires ¹⁴ ¹⁵. For gameplay, we standardize to one drone class for now – a medium-sized drone with both **sensing** (thermal camera) and **suppression** capability. We envision something like a robust twin-engine drone carrying a water tank or retardant payload, with VTOL or STOL capability to operate in various terrain. These drones can travel quickly to a fire and make repeated drops.

Firefighting Payload: Drones can carry **water or fire retardant chemicals** to drop on fires. Real aerial firefighting uses water, foams, gels, and Phos-Chek retardant (the familiar red slurry) ¹⁶ ¹⁷. Phos-Chek (a phosphate fertilizer-based retardant) is effective for creating fire breaks and sticking to vegetation ¹⁷, while water and foam are used directly on flames. To keep it simple, our drones will use **water with a fire-suppressant additive** (implying a foam or gel) for short-range drops, and possibly **retardant bombs** for larger drops if we design multiple drop types. In third-person mode, the player controlling a drone will have a limited number of drops before the drone must **refill** (simulating returning to base or to a nearby “tanker” aircraft). For instance, each drone might carry 1000 liters, and one drop uses 250L – giving four drops per sortie. Dropping effectively on the fire reduces its intensity bar. If the tank runs out, the drone automatically exits the fire zone to “refuel” (we won't show the base; it can despawn or fly off-map and then become available after a cooldown). This encourages the player to cycle between fleets and not micromanage one drone forever.

Unit Attributes: Each drone fleet could have stats like speed, drop power, and turn radius. For now, all fleets share stats, but in future one could differentiate (e.g. heavy drones with larger retardant payload but slower, versus light drones that are fast scouts). Drones will be **resilient** (not easily destroyed) since the scenario doesn't involve hostile agents, but we may simulate that flying through intense smoke or heat can temporarily disable a drone (light damage mechanic), encouraging cautious approach (for example, if a drone is kept too long in a high-intensity fire zone in manual mode, it might overheat and be forced to withdraw). This adds a bit of realism – even unmanned systems have limits in hazardous environments.

Country-Specific Units: To add authenticity, the game ties drone fleets to country defense or firefighting units. When a player selects a country to deploy from, the drones launched will use that country's colors and insignia. This feature is **cosmetic and narrative** – it highlights international cooperation and acknowledges that many nations now deploy drones in wildfire fighting. For example: - Selecting the **USA** might deploy drones labeled as *"California Air National Guard Fire UAVs"* with a grey and red color scheme (military grey with red stripes, since the US military often uses grey UAVs, but fire service uses high-visibility red/white on manned aircraft). The U.S. has indeed used Air National Guard MQ-9 Reaper drones to assist wildfire efforts in California ¹⁴, demonstrating this concept in reality. - **Brazil** could deploy drones from its Army or Air Force for Amazon fire response – colored in olive green or rainforest camouflage, with a Brazilian flag. (Brazil's military has been involved in Amazon firefighting, though primarily with manned planes; in our game we envision a drone unit in their arsenal.) - **Canada** might use a white and red scheme (reflecting Canada's firefighting aircraft livery) or the colors of the Canadian Armed Forces. Canada has utilized drones for fire mapping in Alberta ¹⁸, so it's fitting to include them. - **Australia** could use orange or yellow drones (Australia's Rural Fire Service vehicles are yellow; water-bomber aircraft often have high-visibility schemes). Australia is exploring drones for bushfire management ¹⁹. - **Other examples:** Europe (e.g. France or EU) might have white and blue "EU Fire UAV" units, China might have military-green drones (China is developing autonomous early warning drones for forest fires ²⁰), etc. We include at least the major players in wildfire response.

These distinctions are primarily for immersion. In gameplay terms, all drones behave the same regardless of country. However, one could introduce slight differences in future (e.g. U.S. drones have a marginally faster response time, Australian drones have better endurance, etc.) if balancing required. For the hackathon prototype, we stick to color/name differences as noted by the user's plan.

5. Scoring System

The scoring system is designed to reflect real-world values – rewarding the player for saving lives, property, and land (which aligns with how agencies and defense forces would value mission success). We incorporate both **point scores** and a **monetary value of damage averted** to give tangible feedback.

Key components of the score:

- **Lives Saved:** Each wildfire is assumed to threaten a certain number of people (population in nearby towns, firefighters on the ground, etc.). If the player extinguishes a fire before it spreads to populated areas, those lives are "saved." We will assign an approximate number of lives at risk per fire (based on location and size – e.g. a fire in remote forest might have 0 lives at immediate risk, a fire near a city could have thousands). For scoring, each life saved could be worth a significant score bonus (e.g. 100 points per person). We won't convert lives to dollars (for sensitivity reasons), but in narrative we might say "X lives protected." The game could display a **running counter** of population

saved. *Realism basis:* Wildfires cause fatalities; preventing those is the top priority in real emergency response, so our scoring puts heavy weight on it.

- **Property & Economic Damage Prevented (\$):** As fires are contained, we calculate the estimated property damage that was avoided. For instance, if a wildfire was on track to burn 1000 acres of farmland and perhaps 100 homes, putting it out early saves those assets. We can use rough metrics: in the U.S., wildfires caused about **\$10.2 billion** in property damage annually in recent years ²¹, and the average cost to rebuild a home is hundreds of thousands of dollars ²² ²³. For our game, we might assign a dollar value per acre and per structure saved. Example approach: each acre of forest = \$X value (for timber, environmental value), each home = \$Y value (e.g. \$300k), critical infrastructure (power lines, etc.) even more. At game end, we sum these to report “Estimated Damage Averted: \$ [value]”. This not only acts as a score metric but also educates the player on the immense economic stakes of wildfires. We'll cite that globally wildfires cost **tens of billions** annually in damages ²¹, so even saving a fraction in one scenario is significant.
- **Fire Contained (Objective Score):** Each fire extinguished yields base points. For example, *extinguish a small fire* = 100 points, *medium* = 250, *large* = 500. This encourages players to tackle as many fires as possible. If a fire is only partially contained when time runs out, partial points could be awarded based on how much the player reduced it (though typically it's binary – either put out or not – we might simplify and only award on full extinguish).
- **Time Efficiency Bonus:** Faster response is critical in wildfire fighting. If the player extinguishes all fires before the clock, or contains a fire very quickly after it ignites, they get bonus points. For example, extinguishing a fire within 1 minute of it appearing could give a +50% bonus for that fire's points. If any fires remain at the end, we might deduct points proportional to their size (representing damage done). Essentially, the game favors **quick, strategic action**.
- **Resource Management & Penalties:** We could introduce a minor penalty for inefficient use of resources. For instance, if a player deploys all drones to a single small fire (overkill) while other fires rage, the score might suffer indirectly (because other fires grow – causing higher damage). We won't explicitly subtract points for “wasted water”, but the opportunity cost is the penalty (uncontrolled fires reduce score by causing damage). Additionally, if a drone is **lost** (e.g. the player flies it into the ground or it gets destroyed by extreme conditions), we might subtract a fixed score (to simulate equipment loss and possibly injuries if it were a manned craft). Since these are uncrewed, losing one isn't loss of life, but it is a failure of the mission.

The end-of-round screen will break down the scoring: - Number of fires extinguished, with points from each. - Lives saved (with possibly a narrative note like “Evacuations prevented: 5,000 people”). - Property saved (dollar estimate). - Total score. - A performance rating (e.g. *Mission Impact: High* if lives saved > some threshold, etc.). This can tie back to **defense perspective**: In a real after-action, one might report that our drone intervention saved X million dollars in assets and Y lives – exactly the kind of justification defense agencies use when supporting civilian missions.

We base these calculations on real considerations: government agencies routinely estimate wildfire impact in monetary terms (property, firefighting costs, economic loss from business interruption) ²⁴ ²⁵. For instance, a single large fire in California can cause billions in damage if it reaches urban areas. By including

a dollar figure, the player can intuitively grasp the value of their fast action (e.g. “You saved \$2.5 billion in potential damage by containing the fires early!”).

Technical Implementation

1. Tools & Technologies

- **Unity 6.2 Beta (URP):** We use Unity game engine’s 6.2 beta release, taking advantage of the Universal Render Pipeline for optimized graphics on both PC and WebGL. URP ensures efficient rendering of our large scene (global terrain + multiple moving objects + particle effects) and is easier to deploy to WebGL than the HDRP. We will need to be mindful of any beta instabilities; however, Unity 6.2 offers the latest .NET support which might help in handling JSON data parsing, etc.
- **Programming:** The game is coded in C# within Unity. We’ll have scripts for data fetching (NASA API integration), gameplay logic (drone AI, fire behavior), UI (for scores, timers, and deployment controls), and possibly simple physics for water drops. The team will employ a **modular approach**: e.g., one module handles **Geospatial Mapping** (converting lat/long to world coordinates, integrating Cesium or a custom globe), another handles **Drone Fleet AI** (movement, pathfinding from base to fire), another deals with **Fire Dynamics** (spreading, extinguishing progress).
- **NASA Data Integration (API Documentation):** Using NASA’s FIRMS **Web Services API** requires an Earthdata login and obtaining a Map Key ⁹. We will register and then use the **active fire data endpoint**. For example, to get global fires from the last 24 hours in JSON, the API might be: `https://firms.modaps.eosdis.nasa.gov/api/area/csv/{map_key}/VIIRS_SNPP_NRT/24h/` with additional parameters (this is an illustrative format; actual endpoints are documented by NASA ⁴). The API allows filtering by sensor (MODIS vs VIIRS) and time window. Since near-real-time data is updated at least every 3 hours ²⁶, a single fetch at game start is enough. We will include in our documentation the specific REST call and the fields returned. For example, a sample JSON entry might look like:

```
{
  "latitude": 34.56,
  "longitude": -119.34,
  "brightness": 350.7,
  "datetime": "2025-08-20T14:30:00Z",
  "confidence": "high",
  "surface_type": "vegetation"
}
```

We will document the meaning of key fields (e.g. brightness or Fire Radiative Power indicates intensity; confidence is the algorithm’s confidence in it being a real fire). This information can help in-game (maybe we show high-confidence fires as solid icons, low-confidence as flickering until confirmed). The **Earthdata API usage guidelines** and rate limits will be noted to ensure we don’t exceed them (5000 transactions per 10 minutes ¹¹ is generous for our needs; we’ll likely make 1-2 calls per session). If the API is unreachable (network issues), we’ll have a fallback to load a local dataset (we will prepare a CSV of recent fires as backup).

- **Cesium for Unity (Geospatial Streaming):** If using Cesium, we must install the Cesium for Unity package (open-source). We will include instructions in the technical doc: import the Cesium Unity package, obtain a Cesium ion access token (free community token allows streaming base imagery and terrain), then add a `CesiumGeoreference` and `Cesium3DTilesset` to the scene. We'll cite Cesium's docs: "*Cesium for Unity... enables real-world 3D data to be utilized in Unity*" ⁶. We need to ensure any required plugin DLLs can compile to WebGL (Cesium for Unity uses Cesium Native under the hood; as of 2025 it may support WebGL via WebAssembly, but if not, our WebGL build might need the simpler globe fallback). We will clarify this in tech notes: *If WebGL build is targeted, test Cesium's compatibility; otherwise plan to use static textures*. For the PC build, Cesium streaming will show off high detail (like 3D terrain in California or wherever fires are).
- **Google Maps API (Optional Integration):** Though likely unused, we document that if one wanted to integrate Google Maps data (say to get a detailed satellite image of a particular region or use Google's elevation), one would need to use Google Maps REST APIs. For example, Google's Static Maps API could fetch a map tile given lat/long bounds, or the Elevation API could give terrain heights. However, since **Google's Unity SDK for Maps is proprietary** ¹², we opted not to use it. We note that *if* a developer had access, they could incorporate Google's real-world building models for urban firefighting scenarios – but our focus is wildland fires, so detailed buildings are less crucial.
- **Aerial Firefighting References:** In the technical document, we'll list the references used for realism: e.g., **Wikipedia Aerial Firefighting** for chemicals and tactics ¹⁶ (so developers know what each term means), and **Drones in Wildfire Management** for understanding how drones are used by actual agencies ²⁷. This ensures that any further development can continue in a realistic direction (like possibly adding a surveillance drone type that maps the fire perimeters using IR cameras, as is done in reality ²⁸, which could be a future feature).
- **AI and Behavior:** We'll implement simple AI for drones: e.g. a drone fleet assigned to a fire will move to that location (following a great-circle path on the globe). Unity's NavMesh doesn't apply on a sphere easily, so we'll likely script movement. Drones can slow down as they approach the target. Once over the fire, they will orbit and periodically execute a "drop" action (reducing fire intensity). We might use a particle system for water drop visuals (blue particles extinguishing orange flame particles on the ground). Fire on the ground can be a particle system or textured quad that reduces as intensity goes down.
- **User Interface:** The UI will include a world map HUD (possibly a small minimap or list of largest fires), a timer, a score display (updated in real-time with points and \$ saved), and controls to select countries/drones. For country selection, a simple panel with flags or a dropdown can be used. When a country is picked, the available drone fleet count is shown. The player then clicks a fire marker to dispatch. We'll highlight the fire marker or draw a line from the country to the fire to indicate deployment en route. If the player tries to deploy with no fleets available, an error message will show ("All drone teams are busy!"). During third-person drone control, a crosshair and maybe an altimeter will show on screen, and a "payload remaining" bar to indicate how many drops left.
- **WebGL Deployment:** The aim is to have a **WebGL build** so the game can be embedded on the hackathon site or easily shared. Unity 6.2's WebGL exporter will be used, ensuring we keep assets and shaders WebGL-friendly (URP helps by being WebGL-compatible out of the box). We must keep texture sizes moderate, limit heavy physics or real-time shadows to maintain frame rate in browser. If

Cesium is too heavy for WebGL, as mentioned, we will use a static globe approach for that build. The PC build can have the full feature set (and be used for a high-quality demo video). We'll note in the tech doc any differences in the WebGL version (for instance, we might disable some fancy effects or limit the number of visible fires if performance dictates).

2. Security & Sustainability

Because this project could conceptually be used by defense or government, we consider **security and long-term maintainability**:

- **Data Security**: All NASA data used is public and open, so no sensitive info is involved. We still follow best practices in API usage: we will not expose the Earthdata API key in client-side code for WebGL (to avoid misuse). Possibly, we route data requests through a lightweight proxy or require the player to input their own API key if open-sourcing the project. For the hackathon demo, using our own key embedded is acceptable (with the understanding it's a limited-use key). Earthdata uses HTTPS, ensuring data integrity in transit.
- **Sustainability**: The simulation relies on well-established data sources (NASA Earthdata) which are likely to remain available for years. We document how to update data retrieval if needed (e.g. FIRMS API version changes). The game's modular design means it can be extended – e.g., adding new drone types or integrating new data (like weather data from NASA if we wanted dynamic wind affecting fires). We use Unity, a widely supported engine, and avoid unusual third-party plugins beyond Cesium (which itself is open-source and actively maintained ⁶) and possibly ArcGIS or others if needed (also maintained by Esri). This ensures the project can be maintained or expanded by others (e.g. future hackathon teams or even as a training sim for agencies).
- **DoD Testing Readiness**: If the Department of Defense were to test or adopt this prototype, we highlight that it uses **open-source tools** and **unclassified data** – easing approval. We can package it for Windows which DoD testers can run offline (using archived data). All components (Unity, .NET, Cesium) can be audited. We would include an option to run entirely offline with pre-downloaded fire datasets for secure environments.

3. Team Collaboration

This project can be built by a small team (e.g. 3-5 people) over a few days by dividing tasks: one focuses on **data/API integration**, another on **gameplay logic and drone controls**, another on **UI/UX and scoring**, etc. Unity's scene and prefab system allows parallel work (using source control like Git with Unity YAML serialization). We maintain clear documentation (this design doc doubles as a reference) so each member knows how their part interfaces with others. For example, the person handling NASA data provides an interface like `FireDataManager` with a list of fire locations that the gameplay programmer uses to spawn fires. Good communication and flexibility are key in a hackathon – if a certain API or approach fails, the team is prepared with fallback plans (e.g. if streaming the globe fails, switch to static map). The design emphasizes modularity, meaning one can swap out components (like use Mapbox instead of Cesium for maps, or use a different scoring formula) without rewiring the entire game. This not only helps during development under pressure, but also showcases **teamwork and resilience**, as each member can debug their module while others continue building.

Finally, we plan to record a **gameplay video trailer** showcasing the core features: the Earth with real fire data, deploying drones, first-person drops on a blaze, and the final score screen. This will be used in our hackathon presentation.

Alignment with Judging Criteria

To ensure our project excels in the NDIA Hackathon judging, we map our design to each criterion:

- **Technical Innovation (25%):** Our solution is original in combining *real-time NASA satellite data* with a *gaming simulation*. We stream live wildfire data into Unity, which is technically challenging and innovative – demonstrating how defense simulations can integrate real-world sensor inputs. The use of Cesium 3D globe streaming in a Unity game ², and controlling drone swarms in an RTS/third-person hybrid gameplay, shows depth in technical implementation. We also leverage modern APIs (NASA FIRMS, Cesium ion) in creative ways. This concept could evolve into a serious training tool for coordinating autonomous firefighting units, which is a novel application of both geospatial tech and game engines.
- **Mission Impact (30%):** Wildfires are a national security concern (they strain resources, threaten bases, destabilize regions) – our game directly addresses this by exploring how military drones could assist in disaster response. The scenario aligns with defense priorities of using unmanned systems for humanitarian assistance and disaster relief. By letting the player act as a **global incident commander**, the project highlights inter-agency and international cooperation, a key real-world factor. The realism (actual fire locations, near real-time monitoring) makes the solution relevant to real use cases; for instance, DoD could use such a simulation to strategize deployments of assets during wildfires. This broad *mission impact* is baked into the design – every aspect (drones, data, scoring by lives saved) ties to saving lives and land, showing clear societal and defense value.
- **Usability & Design (20%):** We emphasize a clear, intuitive UI despite the complexity under the hood. The globe interface allows users to literally *see the whole picture* at a glance – fires are clearly marked, and controls for deploying drones are straightforward (select country, click fire). The switch to drone POV is seamless with one button, making it easy even for a novice to jump in and start dropping water. We keep text and visual clutter minimal: important info (timer, score, number of drones left) is displayed in corners, and context-specific hints (like “Click a fire to deploy drones”) guide the player. We’ve kept paragraphs short in documentation and will maintain clean code structure – which translates to maintainable design. The game design also inherently guides the player: blinking icons or alerts draw attention to new fires, etc. The **demo clarity** is high – viewers/judges can quickly grasp the concept because they see fires and drones in action in a familiar Earth visualization. The potential for reuse/extension is strong: the modular design means new scenarios (like other disaster types or expanded drone capabilities) can be built on this framework, indicating a well-thought-out design.
- **Security & Sustainability (15%):** As discussed, our system uses secure, open data APIs and avoids proprietary lock-in (aside from Unity engine itself). NASA Earthdata APIs are robust and widely used ⁹, and the data is free – making long-term maintenance feasible. We handle API keys appropriately and document all external dependencies. Should this project be scaled up, it can integrate with DoD systems because it’s built on standard tech (no unusual hardware or OS requirement; runs on a Windows PC or web browser). The code and data practices follow open-source spirit (e.g. Cesium plugin is open source ⁶, NASA data is open). We also address sustainability in terms of **expandability**: new data layers (like NASA weather or vegetation maps) could be added to increase accuracy of fire behavior, and new drone tech (e.g. AI-driven firebreak

ignition drones) can be simulated. The system could thus remain useful as a research or training tool beyond the hackathon.

- **Team Collaboration (10%):** Our approach to development is collaborative and resilient. We've clearly delineated roles and system components so a team can work in parallel without blocking each other. For instance, while one member sets up the globe and NASA data feed, another can prototype drone flight controls with placeholder targets. We also planned fallback options (like if live data fails, use offline data; if Cesium fails, use static map) – this contingency planning is evidence of a team that can handle pressure and adapt. The final presentation (with the video and this comprehensive document) will be polished and well-coordinated, reflecting strong teamwork. We will also mention each contributor's part, showing how we integrated everyone's strengths.

Conclusion

Wildfire Drone Deployment Simulation is not just a game; it's a convergence of real-world data, defense technology, and interactive simulation. By conducting deep research into NASA's fire data APIs, mapping tools, and firefighting practices, we have crafted a design that is both achievable in a hackathon sprint and impressive in its scope. This Game Design Document and Technical Plan will guide development over the next 1–3 days, ensuring we hit the ground running with data integration and core gameplay. The end result will be a compelling prototype aligning with NDIA's themes – one that judges can *play* or watch and immediately appreciate its innovation, impact, and potential.

We are confident that with this plan, our team can deliver a prototype that scores highly on all criteria and, more importantly, sparks ideas for how the defense community might use game technology and real data to tackle natural disasters in the future.

Sources:

- NASA FIRMS (Fire Information for Resource Management System) – provides near real-time global fire data (updated within 3 hours of satellite observation) ¹. Data available via API in CSV/KML/JSON formats for the last 24/48 hours or 7 days ⁴.
- NASA Earthdata API documentation – requires a free API key (MAP_KEY) for FIRMS web services, with usage limits (5000 transactions per 10 minutes) ⁹ ¹¹.
- Cesium for Unity – open source Unity plugin enabling streaming of a full-scale WGS84 3D globe with high-resolution terrain and imagery ² ⁶. Integrates with Unity scene for georeferenced content.
- Google Maps Unity SDK – existed as a proprietary SDK for real-world maps in Unity; not publicly free ¹². Alternative mapping solutions (Mapbox, ArcGIS Maps SDK ¹³) were noted for completeness.
- Aerial Firefighting Techniques – aircraft use water, foam, gel, and Phos-Chek retardant to combat fires ¹⁶. Phos-Chek is a common fertilizer-based retardant dropped in wildfires ¹⁷.
- Drones in Wildfire Management – many countries (USA, Canada, Australia, Europe, etc.) are increasingly using drones for wildfire surveillance and suppression ²⁹. U.S. agencies have deployed UAVs like NASA's Ikhana (Predator drone) and National Guard MQ-9 Reapers in wildfire response ¹⁴ ¹⁵. Canada and Netherlands have run drone operations for fire mapping and early detection ¹⁸ ³⁰.
- Wildfire Impact Statistics – Wildfires cause immense damage; e.g., an estimated **\$10.2 billion** in property damage in recent years in the U.S. ²¹. Millions of homes are at risk, with hundreds of

billions in property value exposed to wildfires ²² ²³ . Effective firefighting can save significant economic value and lives.

¹ ²⁶ **FIRMS | NASA Earthdata**

<https://www.earthdata.nasa.gov/data/tools/firms>

² ⁶ **Cesium announces a free, open source plugin for the Unity game engine - Geoawesome**

<https://geoawesome.com/cesium-announces-a-free-open-source-plugin-for-the-unity-game-engine/>

³ **Unity with MapTiler real-world 3D content**

<https://docs.maptiler.com/unity/>

⁴ **NASA FIRMS - Active Fire — pyro_risks 0.1.0a0-git documentation**

https://pyronear.org/pyro-risks/overview/datasets/NASA-FIRMS_ACTIVE-FIRE_VIIRS.html

⁵ **Articles tagged as Drones | Smithsonian Magazine**

<https://www.smithsonianmag.com/tag/drones/?page=4>

⁷ **Global Imagery Browse Services APIs - NASA Earthdata**

<https://www.earthdata.nasa.gov/engage/open-data-services-software/earthdata-developer-portal/gibs-api>

⁸ **NASA's Global Imagery Browse Services (GIBS) Provides Access to ...**

<https://cesium.com/blog/2015/06/30/nasa-gibs>

⁹ ¹⁰ ¹¹ **API - Data Availability - NASA | LANCE | FIRMS**

https://firms.modaps.eosdis.nasa.gov/api/data_availability/

¹² **Is there a Google SDK for rendering Google Maps in Unity?**

<https://stackoverflow.com/questions/51684525/is-there-a-google-sdk-for-rendering-google-maps-in-unity>

¹³ **ArcGIS Maps SDK for Unity | Esri Developer**

<https://developers.arcgis.com/unity/>

¹⁴ ¹⁵ ¹⁸ ²⁰ ²⁷ ²⁸ ²⁹ ³⁰ **Drones in wildfire management - Wikipedia**

https://en.wikipedia.org/wiki/Drones_in_wildfire_management

¹⁶ **Aerial firefighting - Wikipedia**

https://en.wikipedia.org/wiki/Aerial_firefighting

¹⁷ **This Is What Is In The Red Water Dropped By Firefighting Planes**

<https://www.jalopnik.com/this-is-what-is-in-the-red-water-dropped-by-firefightin-1851736822/>

¹⁹ **Australia: High-tech drones may soon be fighting bushfires**

<https://www.preventionweb.net/news/its-science-not-fiction-high-tech-drones-may-soon-be-fighting-bushfires-australia>

²¹ **jec.senate.gov**

https://www.jec.senate.gov/public/_cache/files/9220abde-7b60-4d05-ba0a-8cc20df44c7d/jec-report-on-total-costs-of-wildfires.pdf

²² ²³ **Wildfire Damage & Home Insurance Claims by State - ValuePenguin**

<https://www.valuepenguin.com/homeowners-insurance/wildfire-statistics>

²⁴ **Climate-exacerbated wildfires cost the U.S. between \$394 to \$893 ...**

<https://www.jec.senate.gov/public/index.cfm/democrats/2023/10/climate-exacerbated-wildfires-cost-the-u-s-between-394-to-893-billion-each-year-in-economic-costs-and-damages>

25 Wildfires are deadlier and costlier due to climate change

<https://hsph.harvard.edu/climate-health-c-change/news/wildfires-are-deadlier-and-costlier-due-to-climate-change/>